Generative AI Systems Design Interview
101 Lessons

1 Introduction and Overview
2 Ride-Sharing Company
3 Image Tagging
4 ChatGPT Focused Interview Prep
5 Image Captioning
6 Machine Augmented Generation
7 Realtime Price Generation
8 High Resolution Image Synthesis
9 Text to Image Generation
10 Personalized Recommendation Systems
11 Text to Video Generation

08
High-Resolution Image Synthesis

Introduction

Generative AI offers a fascinating ability to create highly realistic and diverse images. In this chapter, we explore a technique that enables the generation of detailed and vivid images in just seconds.

Figure 1: An image generated by Midjourney.

Clarifying Requirements

Here's a typical interaction between a candidate and an interviewer.

Candidate: Should the system focus on specific categories or image types?

Interviewer: For simplicity, let's begin with natural scenes and other landscapes. We can explore other categories later.

Candidate: Do we need to ensure the generated images are unique or can we produce variations?

Interviewer: We want to strike a balance between high resolution and high diversity in terms of shapes and landscapes.

Candidate: Should the system support additional conditioning, such as input text descriptions, for more detailed control?

Interviewer: Great question. We'll focus on class-conditional and text-to-image input conditions. However, the system should be flexible to support input extensions.

Candidate: Do we need to ensure image should up and be upfront generating the image?

Interviewer: Yes, we want the system to generate images within a few seconds, ideally within 5 seconds for a single image.

Candidate: Should the system be open-sourced, or is it a closed, fully accessible?

Interviewer: Keep this strictly an in-house system. However, it is reasonable prototyping time to implement. Let's aim for first accurate version in two weeks.

Frame the Problem as an ML Task

Specifying the system's input and output

For high-resolution image synthesis, the user simply requests a text image. The output is a high-resolution image.

Figure 2: Input and output of an image generation system.

Choosing a suitable ML approach

As discussed in Chapter 7, there are several approaches to image generation, including VAEs, GANs, autoregressive models, and diffusion models. In this section, we discuss the best suited for the task.

Most variants of VAEs and GANs struggle to generate high-resolution images, such as those with resolutions of 512x512 pixels and above. They face a challenge known as posterior collapse, where the model fails to capture the distribution increases, these models require a decoder with a higher capacity to capture additional details. During training, the decoder can become so powerful that it starts ignoring the latent variables, relying instead on its own capacity to generate outputs. This results in a disconnected variables contribute little to the generated outcome, limiting the diversity of the generated images.

While both autoregressive and diffusion models can generate high-resolution images, they differ significantly in complexity and efficiency. Autoregressive models treat image generation as a sequence prediction task, where each pixel is predicted based on the pixels that came before it. This dependency leads to a slow process that increases linearly with the number of pixels, resulting in an O(N²) complexity for an image of size N × N, and the process is difficult to parallelize. To address this limitation, some autoregressive models operate in a compressed latent space, such as those used in VQ-VAE-2 or DALL-E. However, even with this approach, sampling times of 20-60 seconds remains slow. In contrast, diffusion models, while slower than GANs, significantly reducing the computational overhead compared to traditional pixel-based methods.

On the other hand, the complexity of diffusion models increases super-linearly with image size, resulting in a computational complexity of O(N² log N), where N is the number of pixels and T represents the number of denoising steps. Larger images often require more denoising steps to maintain quality, further exacerbating the computational demands and slow inference times.

In summary, generating a high-resolution image using standard diffusion models can take several minutes. In contrast, Transformer-based autoregressive models can accelerated certain tasks to seconds due to their pixel-based generation approach. For this reason, we'll focus on Transformer-based autoregressive models for our discussion. In Chapter 9, we will cover diffusion models in detail. Let's now delve into the autoregressive models and their key components.

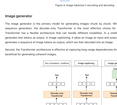


Figure 3: Autoregressive image generation.

Autoregressive models generate images by treating them as a sequence generation task. This approach relies on two primary components:

- Image tokenizer
- Image generator

Image tokenizer

Image tokenizers refer to representing an image with a sequence of discrete tokens. This is crucial in autoregressive models, where the image is generated sequentially, chunk by chunk.

The image tokenizer is a crucial component in autoregressive models, as it enables us to encode an image into a sequence of discrete tokens and decode a sequence of discrete tokens back into an image.

Figure 4: Discrete Unity Transformer's flexibility in handling various modalities.

In summary, an approach image generation involves two key components: tokenizer and transformer. For an image generation system, the tokenizer plays a crucial role in encoding images into a sequence of discrete tokens and decoding these tokens back into images. We will explore the architecture, training, and sampling processes of these components in detail in the model development section.

Data Preparation

The data preparation process involves two crucial steps:

- Image tokenizer preparation
- Image transformer

Image cleaning and normalization

At this step, we remove low quality images from the training data and ensure the remaining ones are consistent. This is achieved by applying the following operations:

- **Remove low quality images:** We remove images with low resolution, excessive noise, or irrelevant content. We also ensure the dataset includes a diverse range of subjects, styles, and compositions. This ensures the model learns to generate high-quality, diverse, high-quality images.
- **Normalize images:** Normalization involves scaling pixel values to a range, typically 0 to 1, to stabilize the training process. We also standardize the image dimensions, resizing and cropping images to a fixed resolution if necessary, ensuring compatibility with the tokenizer's expected inputs. Based on the tokenizer's requirements, we resize all images to 256x256.

Image tokenization

The image generator requires images to be represented as a sequence of discrete tokens. To achieve this, after training the image tokenizer, we use it to tokenize all images in our dataset. This process involves the following steps: It is important to note that this data preparation step is intended primarily for the image generator, not the image tokenizer.

Figure 7: Data preparation process.

These two steps ensure the training data is high-quality, consistent, and represented as a sequence of numerical tokens.

Model Development

Architecture

In this section, we explore the architecture of both the image tokenizer and the image generator.

Image tokenizer

The image tokenizer model has two functions:

1. Encoding an image into a sequence of discrete tokens.
2. Decoding a sequence of discrete tokens back into an image.

A common architecture employed for image tokenization is the Vector Quantized VAE (VQ-VAE) [3], which is a variant of the standard VAE discussed in Chapter 7. The VQ-VAE consists of three components:

- Encoder
- Quantizer
- Decoder

Encoder

The encoder maps the input image into a lower-dimensional latent space. This component encodes important features of the image while discarding less relevant details.

The encoder's architecture is a deep convolutional neural network (CNN) with several convolution layers, each followed by a ReLU activation function. These layers progressively reduce the spatial dimensions of the input image.

Figure 8: The encoder converts an input image into an encoded representation combining the context, each with a channel.

Quantizer

The quantizer converts continuous latent vectors into discrete tokens. These are two main reasons why VQ-VAE introduces a quantizer component to a standard VAE:

- Preventing posterior collapse
- Making the model suitable for generation

Avoiding posterior collapse

Posterior collapse is a common issue in standard VAEs where the latent variables contribute little or are ignored because the decoder becomes too powerful and learns to reconstruct the input without relying on the latent variables. By introducing a discrete latent space, the model is forced to learn meaningful representations. The decoder doesn't overpower the latent space and must leave the latent variables actively involved in shaping the output.

Making the learning tractable

Continuous vectors are difficult to predict sequentially because they have endless possibilities and small differences. By turning these vectors into discrete tokens, the quantizer simplifies the problem by allowing the model to concentrate on focus on fewer options.

The quantizer maps the encoder's continuous latent vectors to discrete tokens. The codebook contains a fixed number of embedding vectors, each representing a distinct token. The quantizer finds the closest embedding in the codebook for each latent vector. The image from 1 to N. The quantizer replaces each continuous vector with the closest token in the codebook based on Euclidean distance [4].

Figure 9: Quantization process.

Note that the quantizer is an embedding table, its sole parameter is the codebook, which is learned during training. The quantization process simply involves finding the nearest embedding vector within the closest token in the codebook. Therefore, the output is a collection of token IDs.

Decoder

The decoder converts discrete tokens back into the original image. It typically uses a deep CNN with transposed convolutions, mirroring the encoder's architecture. The goal for the reconstruction is the original image from the codebook. Therefore, the output is a collection of pixel values.

Figure 10: Decoder step. Transformer components.

Training

A autoregressive image generation, we have two training stages.

- Stage 1: Training the image tokenizer
- Stage 2: Training the image generator

Stage 1: Training the image tokenizer

The training process involves optimizing the encoder, decoder, and codebook as the model can accurately reconstruct the original images. The process can be described in three steps:

1. The encoder maps the input image into a continuous latent representation.
2. The quantizer replaces the continuous representation with discrete tokens using its learned codebook.
3. The decoder uses the discrete tokens to reconstruct the original image.

Figure 11: Image tokenizer training process.

Since the quantizer block operates like a self-attention gradient for backpropagation, the VQ-VAE paper proposes approximating the gradient by copying it from the decoder input directly to the encoder output. The approach means that only the selected tokens receive gradient updates, while the unused codebook embeddings do not receive any gradients.

Stage 2: Training the image generator

The training objective for the image generator is to predict the next token in a sequence. Given a sequence of tokens, the model learns to predict the probability distribution for the next token. In particular, we use the Cross-Entropy (CE) loss, which contrasts with softer images. This results in a more calibrated that captures diverse visual patterns.

Inference and Sampling

The CE objective of the image tokenizer is accurately reconstruct original images from their quantized tokens. To achieve this, the decoder, the Transformer-like functions are typically employed during the inference process.

- Reconstruction loss.
- Quantization loss.

Reconstruction loss: The reconstruction loss measures the difference between the original image and its reconstruction from the quantized tokens. It is typically calculated using the mean squared error (MSE) loss:

$$\text{Reconstruction loss} = \|x - \hat{x}\|^2$$

Where:

- x is the pixel value of the original image.
- $\hat{x}$ is the pixel value of the reconstructed image.
- $\|\cdot\|$ is the Euclidean norm of the difference.

Quantization loss: This quantization loss measures the distance between the encoder's outputs and the nearest embedding in the codebook. The loss encourages the encoder to produce outputs that are close to the codebook vectors.

$$\text{Quantization loss} = \|sg[z(x)] - e\|^2 + \beta\|z(x) - sg[e]\|^2$$

Where:

- $z(x)$ is the continuous latent vector produced by the encoder, E, from the input x.
- e is the quantized latent vector selected from the codebook, Q.
- $sg[\cdot]$ represents the stop gradient operation that blocks the gradients from flowing through the term. It is used here to prevent the codebook from being updated when minimizing the loss.

For more detailed on the Quantization loss formula, refer to the VQ-VAE paper [3].

Total loss: The total loss combines the reconstruction loss and the quantization loss. Training solely said for reconstructing low-resolution images. However, for high-resolution images, the model may prioritize artifacts. To improve reconstruction quality of high resolutions, two additional loss functions are typically employed:

- Perceptual loss
- Adversarial loss

Perceptual loss: Perceptual loss measures the difference between the features of the original and reconstructed images extracted from a specific layer of a pre-trained model such as VGG [5]. The formula is:

$$\text{Perceptual loss} = \|\phi_j(x) - \phi_j(\hat{x})\|^2$$

Where:

- ϕ_j denotes the feature map of the layer j, from a pre-trained VGG model.
- x is the original image.
- $\hat{x}$ is the reconstructed image.

Perceptual loss encourages the model to reconstruct images that are perceptually similar to the original images. GAN features include high-level details such as content and style. The perceptual loss guides the training process so that the model can better capture these crucial details in reconstructed images.

Adversarial loss: Adversarial loss is derived from GAN [6], designed to minimize differences between real and reconstructed images. It employs a discriminator network to distinguish between real and reconstructed images. The adversarial loss is then calculated based on the discriminator's output. The formula is:

$$\text{Adversarial loss} = -\log(D(\hat{x}))$$

Where:

- D is the discriminator network.
- $\hat{x}$ is the reconstructed image.

This loss function encourages the model to produce reconstructed images that a trained discriminator cannot distinguish from real images. The GAN loss paper introduced a parallel trained version of the loss to reduce unwanted artifacts and improve the realism of generated images.

Overall loss: The overall loss function is often a weighted sum of the individual losses described above. The weights [λ] are hyperparameters that need tuning to specific architectures, goals and datasets.

$$\text{Overall loss} = \lambda_{\text{reconstruction loss}} \times \text{reconstruction loss} + \lambda_{\text{quantization loss}} \times \text{quantization loss} + \lambda_{\text{perceptual loss}} \times \text{perceptual loss} + \lambda_{\text{adversarial loss}} \times \text{adversarial loss}$$

After training the image tokenizer, we can train a GPT-style model to predict the discrete tokens and cache them, as detailed in the data preparation section. The cross-ensure that of image representations to a sequence of discrete tokens, which is required for training the image generator.

Figure 12: Image tokenizer training process.

Sampling

A autoregressive models, generating a new image involves two steps:

1. Generating a sequence of discrete tokens
2. Decoding discrete tokens into an image

1. Generating a sequence of discrete tokens

In the first step, the image generator produces a sequence of tokens. The autoregressive nature of the generation ensures that each token is conditioned on preceding tokens, leading to coherent outputs.

Figure 13: Sampling process.

Here's a step by step process to autoregressively generate a sequence of tokens:

1. Randomly select a token from the codebook as the first token. This token serves as a seed for the rest of the generation process.
2. Autoregressively generate tokens one by one. This involves:
   1. Feeding the current sequence of tokens to the image generator.
   2. The image generator outputs a probability distribution over the codebook, indicating the probability of each token being the next token.
   3. Sampling the chosen token to the current sequence.

This process continues until the entire image is generated. The number of iterations depends on the resolution and size of the tokenizer's latent space. For instance, with a 256x256 image and a tokenizer that maps it to a 16x16 latent space, we need to sample 256 tokens. Thus, assume 256 tokens. The process continues until all 256 tokens are generated. Once the sequence of tokens is complete, it is transformed into an actual image, which is the focus of the next step.

2. Decoding discrete tokens into an image

In this step, the sequence of discrete tokens is transformed into an image by using the decoding functionality of the image tokenizer.

Figure 14: Decoding tokens into an image.

Evaluation

The evaluation metrics for high-resolution image synthesis are similar to those in Chapter 7. We'll briefly review them in this section without going into detail.

Offline evaluation metrics

The following metrics are typically employed to measure the quality and diversity of the generated images:

- **Inception score:** Measures how similar the generated images are to images of real-world objects by utilizing a pretrained *Inception v3* model. To learn more about the Inception score, refer to [9].
- **Fréchet inception distance (FID):** Compares the distribution of generated images to real images by comparing features extracted from a pretrained *Inception v3* model. This metric measures how similar the statistics of generated and real images are. To learn more about FID, refer to [10].
- **Human evaluation:** Human evaluators are presented with pairs of images and asked to judge their photorealism and aesthetic qualities. The votes provide a statistical measure of which models produce more realistic images over time.

In addition to those metrics, it is common to evaluate other aspects of the model such as latency and cost.

- **Time to generate an image:** Measures the time it takes for the model to generate an image. This metric is important to monitor since users generally expect quick results.
- **Cost per generation:** Calculates the cost to generate an image. This metric depends on factors such as model complexity, resolution, and infrastructure expenses. Monitoring the cost of generation is crucial as it impacts business revenue.

Online evaluation metrics

In practice, companies monitor various metrics to assess the system's real-time quality. Common metrics include:

- **User feedback:** Collects direct feedback from users regarding generated images.
- **Periodic surveys:** Gathers user opinions on the quality and relevance of generated images.
- **Subscription rate:** Measures how often users subscribe to services or features related to image generation.
- **Churn rate**: Measures the rate at which users stop using the service.

Overall ML System Design

Once we are satisfied with the performance of the image generator and image tokenizer models, we can integrate them to construct the image synthesis system. The primary components in a high-resolution image synthesis system are:

- Generation service
- Decoding service
- Super-resolution service

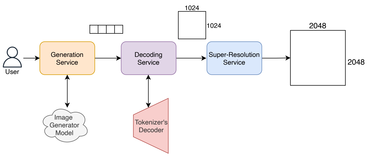


Figure 16: High-resolution image synthesis ML design

Understanding the purpose of each component and their interactions will provide a holistic view of the system. Let's explore each in more detail.

Generation service

The generation service handles user requests and interacts with the trained image generator model to produce a sequence of visual tokens.

Decoding service

The decoding service interacts with the image tokenizer to convert the generated sequence of visual tokens into an image. Note that when we deploy the model, we don't need the encoder in the image tokenizer – it is only used during training.

Separating generation and decoding services is crucial because the image generator and tokenizer are different models with distinct computational needs and latencies. This approach allows each service to scale independently and manage resources efficiently.

Super-resolution service

Super-resolution service uses a pretrained model to increase the resolution of generated images. For example, if the desired resolution is 2048×2048 but the generator produces only 1024×1024, we use a super-resolution model with a 2x upscale factor.

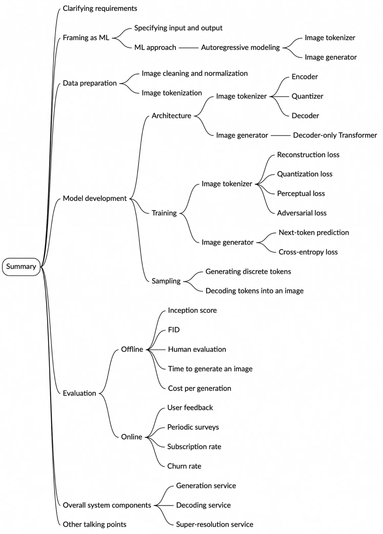
This service is crucial for applications requiring detailed and realistic visuals, such as medical imaging. There are many established solutions for super-resolution, from CNN-based [11] to GAN-improved [12]. To learn more about recent approaches, refer to [13].

Other Talking Points

If there's time remaining at the end of the interview, you could explore these additional points:

- Extending autoregressive models to support text-based generation [14] [15].
- Support applications such as image completion and image super-resolution [16].
- Balancing diversity vs. fidelity in sampling, using techniques such as temperature scaling [17].
- Enhancing the stability with adversarial training, gradient clipping, and learning rate scheduling [18][19].
- Using progressive growing and multi-scale architectures to improve image quality and detail [20].
- Creating interactive systems for users to refine and customize generated images [21].

Summary



Reference Material

[1] Taming Transformers for High-Resolution Image Synthesis. https://arxiv.org/abs/2012.09841.
[2] Neural Discrete Representation Learning. https://arxiv.org/abs/1711.00937.
[3] Deep Learning using Rectified Linear Units (ReLU). https://arxiv.org/abs/1803.08375.
[4] Euclidean distance. https://en.wikipedia.org/wiki/Euclidean_distance.
[5] A guide to convolution arithmetic for deep learning. https://arxiv.org/abs/1603.07285.
[6] LAION data set 400 million https://laion.ai/blog/laion-400-open-dataset/.
[7] Very Deep Convolutional Networks for Large-Scale Image Recognition. https://arxiv.org/abs/1409.1556.
[8] Generative Adversarial Networks. https://arxiv.org/abs/1406.2661.
[9] Inception score. https://en.wikipedia.org/wiki/Inception_score.
[10] FID calculation. https://en.wikipedia.org/wiki/Fr%C3%A9chet_inception_distance.
[11] Image Super-Resolution Using Very Deep Residual Channel Attention Networks. https://arxiv.org/abs/1807.02758.
[12] ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks. https://arxiv.org/abs/1809.00219.
[13] NTIRE 2024 Challenge on Image Super-Resolution (×4): Methods and Results. https://arxiv.org/abs/2404.09790.
[14] Muse: Text-To-Image Generation via Masked Generative Transformers. https://arxiv.org/abs/2301.00704.
[15] VQGAN-CLIP: Open Domain Image Generation and Editing with Natural Language Guidance. https://arxiv.org/abs/2204.08583.
[16] LAR-SR: A Local Autoregressive Model for Image Super-Resolution. https://openaccess.thecvf.com/content/CVPR2022/papers/Guo_LAR-SR_A_Local_Autoregressive_Model_for_Image_Super-Resolution_CVPR_2022_paper.pdf.
[17] Long Horizon Temperature Scaling. https://arxiv.org/abs/2302.03686.
[18] Learning Rate Scheduling. https://d2l.ai/chapter_optimization/lr-scheduler.html.
[19] Adversarial Training. https://adversarial-ml-tutorial.org/adversarial_training/.
[20] Progressive Growing of GANs for Improved Quality, Stability, and Variation. https://arxiv.org/abs/1710.10196.
[21] CogView2: Faster and Better Text-to-Image Generation via Hierarchical Transformers. https://arxiv.org/abs/2204.14217.

Footnotes

1. Certain optimizations and techniques (e.g., latent diffusion model) can significantly speed up the generation process in diffusion models. These methods are discussed in detail in Chapter 10 and Chapter 11. ↩

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.